

Concurrence C++

Threads, Synchronisation, Async, Job System

Adam HAMOUCH

May 2026

1. Threads

Un thread = un fil d'exécution indépendant. Par défaut tout tourne sur le main thread. Le CPU peut en exécuter plusieurs en parallèle (un par core physique).

join() : attend la fin du thread avant de continuer — obligatoire, détruire un `std::thread` encore actif termine le programme.
detach() : se désavoue du thread, il tourne seul — dangereux si le thread accède à des variables locales qui peuvent mourir.

```
std::thread t([]{ uploadToGPU(mesh); });  
t.join();    // attend la fin  
// t.detach(); // laisse tourner seul -- dangereux
```

2. Data Race & Mutex

Data race : deux threads écrivent la même variable simultanément → comportement indéfini. Ex : deux threads incrémentent un compteur → valeur finale imprédictible.

Mutex = verrou. Un seul thread peut le tenir. Les autres sont bloqués jusqu'à sa libération. Le mutex ne protège pas les variables techniquement — c'est une convention : "je prends ce mutex avant de toucher cette donnée".

lock_guard : RAII, libère automatiquement à la fin du scope.

unique_lock : RAII mais peut libérer manuellement — requis par `condition_variable`.

scoped_lock : prend plusieurs mutex d'un coup, anti-deadlock.

```
std::mutex mtx;  
int counter = 0;  
  
void increment() {  
    std::lock_guard<std::mutex> lock(mtx); // prend le verrou  
    counter++;  
} // mutex libère automatiquement ici  
  
// Section critique explicite via accolades  
void render() {  
    auto drawCalls = buildDrawCalls(); // hors section critique  
    {  
        std::lock_guard<std::mutex> lock(mtx);  
        queue.push(drawCalls); // section critique minimale  
    } // mutex libère ici  
}
```

Deadlock : Thread 1 tient A et veut B. Thread 2 tient B et veut A. Les deux attendent indéfiniment → programme figé.
Solution : toujours acquies les mutex dans le même ordre, ou `std::scoped_lock(mtxA, mtxB)`.

3. std::atomic

Opérations atomiques via instructions hardware — pas de blocage de thread, pas de context switch. À privilégier pour une seule variable / opération simple.

Limite : protège chaque variable individuellement mais ne garantit pas l'ordre entre plusieurs atomics.

```
std::atomic<int> counter{0};
std::atomic<bool> ready{false};

counter++; // increment atomique
counter.fetch_add(1); // equivalent
int v = counter.load(); // lecture atomique
counter.store(42); // ecriture atomique
counter.compare_exchange_strong(expected, newVal); // CAS
```

Memory Orders

Le CPU et le compilateur reordonnent les instructions pour optimiser. En multithreading, un thread peut voir les écritures d'un autre dans le mauvais ordre. Les memory orders contrôlent ces garanties.

```
// seq_cst (default) : ordre total garanti pour tous les threads
// Le plus safe, le plus lent (~20-50 cycles de barriere)
ready.store(true); // seq_cst implicite

// release / acquire : le duo classique
// release : tout ce qui precede est visible par celui qui fait acquire
// acquire : voit tout ce qui precede le release correspondant
data = 42;
ready.store(true, std::memory_order_release); // data visible avant ready
if (ready.load(std::memory_order_acquire))
    use(data); // garanti de voir data = 42

// relaxed : atomicite uniquement, aucune garantie d'ordre
// Pour les compteurs de stats ou l'ordre ne compte pas
drawCallCount.fetch_add(1, std::memory_order_relaxed);
```

Regle pratique : par défaut `seq_cst`, optimiser vers `acquire/release` uniquement si mesure comme bottleneck. `relaxed` uniquement pour les compteurs de stats.

4. std::async & Futures

`std::async` execute une fonction sur un thread sans le gerer manuellement et retourne un `std::future` — le receptacle du resultat.
`get()` : bloque jusqu'a la fin et retourne le resultat. Rethrow les exceptions. Invalide le future apres appel.
`wait()` : bloque jusqu'a la fin sans retourner le resultat.
`wait_for(duration)` : attend avec timeout, retourne `ready` / `timeout` / `deferred`.

```
// Toujours specifier la launch policy explicitement
auto fut = std::async(std::launch::async, computeWork); // autre thread garanti
// auto fut = std::async(std::launch::deferred, f); // execute a get()/wait()
// auto fut = std::async(f); // EVITER : async ou deferred, runtime choisit

int result = fut.get(); // bloque, retourne, invalide le future

// wait_for : non-bloquant avec timeout
if (fut.wait_for(0ms) == std::future_status::deferred)
    fut.get(); // deferred : declenche l'execution
else {
    while (fut.wait_for(100ms) != std::future_status::ready)
        doOtherWork(); // async : boucle safe
}
```

std::promise — canal bidirectionnel

```
std::promise<Texture*> p;
std::future<Texture*> fut = p.get_future();

std::thread loader([&p]{
    Texture* tex = loadFromDisk("rock.png");
    p.set_value(tex); // envoie le resultat
});

Texture* tex = fut.get(); // recoit le resultat
```

```
loader.join();
```

std::shared_future — plusieurs consommateurs

std::future = unique_ptr : un seul owner, get() une seule fois.
std::shared_future = shared_ptr : plusieurs threads peuvent appeler get().

```
std::shared_future<Texture*> sf =
    std::async(std::launch::async, loadTexture).share();

// Plusieurs threads peuvent get() independamment
std::thread t1([sf]() mutable { bindAlbedo(sf.get()); });
std::thread t2([sf]() mutable { buildShadow(sf.get()); });
// Utile pour exprimer des dependances entre jobs
```

5. std::condition_variable

Permet a un thread de s'endormir vraiment (zero CPU) jusqu'a ce qu'une condition soit vraie, puis d'etre reveille par un autre thread.

wait(lock, lambda) : libere le mutex, endort le thread, reveille sur **notify**, verifie la lambda — si fausse se rendort (protege contre les spurious wakeups).

notify_one() : reveille un thread en attente.

notify_all() : reveille tous les threads en attente.

Necessite **unique_lock** (pas **lock_guard**) car **wait()** doit pouvoir liberer le mutex pendant le sommeil.

```
std::mutex          mtx;
std::condition_variable cv;
bool                textureReady = false;
Texture*            tex = nullptr;

// Thread de chargement
void loadingThread() {
    tex = loadFromDisk("rock.png");
    { std::lock_guard lock(mtx); textureReady = true; }
    cv.notify_one(); // reveille le thread de rendu
}

// Thread de rendu
void renderThread() {
    std::unique_lock lock(mtx);
    cv.wait(lock, []{ return textureReady; }); // dort jusqu'a textureReady
    draw(tex); // garanti d'avoir la texture
}
```

6. Job System — Thread Pool

Au lieu de creer un thread par tache (couteux : allocation stack, appel OS), un pool fixe de N threads pioche dans une queue de jobs.

3 composants : **queue de jobs**, **worker threads**, **condition_variable** pour le reveil.

```
class ThreadPool {
public:
    ThreadPool(size_t n) {
        for (size_t i = 0; i < n; i++)
            workers.emplace_back([this]{ workerLoop(); });
    }

    ~ThreadPool() {
        { std::lock_guard lock(mtx); stopping = true; }
        cv.notify_all();
        for (auto& t : workers) t.join();
    }

    void submit(std::function<void()> job) {
        { std::lock_guard lock(mtx); jobs.push(job); }
        cv.notify_one();
    }
}
```

```

private:
    void workerLoop() {
        while (true) {
            std::function<void()> job;
            {
                std::unique_lock lock(mtx);
                cv.wait(lock, [this]{ return !jobs.empty() || stopping; });
                if (stopping && jobs.empty()) return;
                job = jobs.front(); jobs.pop();
            } // mutex libere AVANT l'execution du job
            job(); // tous les workers tournent en parallele
        }

        std::vector<std::thread>      workers;
        std::queue<std::function<void()>> jobs;
        std::mutex                   mtx;
        std::condition_variable       cv;
        bool                         stopping = false;
};

// Usage engine
ThreadPool pool(std::thread::hardware_concurrency());
for (auto& entity : entities)
    pool.submit([&entity, dt]{ entity.updatePhysics(dt); });

```

Le mutex protege uniquement la queue. Le job lui-meme s'execute hors section critique — tous les workers tournent en parallele.

7. Double Buffer — Game & Render Thread

Game thread et render thread lisent/écrivent les memes donnees. Un mutex forcerait l'attente et annulerait le benefice du parallelisme.

Solution : deux copies des donnees. Game thread ecrit dans buffer A, render thread lit buffer B. Swap a chaque fin de frame. Jamais le meme buffer en meme temps — zero mutex, zero attente.

Frame lag : le render thread affiche toujours une frame de retard. Imperceptible a 60fps (16ms). C'est le compromis assume par tous les moteurs modernes (Unreal inclus).

```

struct WorldState {
    vec3  positions[MAX_ENTITIES];
    quat  rotations[MAX_ENTITIES];
};

WorldState      buffers[2];
std::atomic<int> writeIdx{0}; // game thread ecrit ici
std::atomic<int> readIdx{1};  // render thread lit ici
std::atomic<bool> frameReady{false};

// Game thread
void gameThread(float dt) {
    WorldState& write = buffers[writeIdx.load(memory_order_relaxed)];
    for (int i = 0; i < count; i++)
        write.positions[i] += velocities[i] * dt;
    frameReady.store(true, memory_order_release); // tout ce qui precede est visible
}

// Render thread
void renderThread() {
    while (!frameReady.load(memory_order_acquire)) {} // attend game thread
    frameReady.store(false, memory_order_relaxed);

    WorldState& read = buffers[readIdx.load(memory_order_relaxed)];
    for (int i = 0; i < count; i++)
        renderer.draw(read.positions[i], read.rotations[i]);

    // Swap des buffers apres le rendu
    int w = writeIdx.load(); int r = readIdx.load();
    writeIdx.store(r, memory_order_release);
    readIdx.store(w, memory_order_release);
}

```

Quelle synchronisation pour quelles donnees

Type de donnee	Synchronisation
Transforms, positions (hot, frame)	Double buffer
Meshes, textures (immutable apres load)	Aucune — lecture libre
Stats, compteurs	atomic + relaxed
Config, asset registry (modifications rares)	mutex
Communication inter-threads	Command queue (pattern Command)